AXIOM: A Shared Controller for Scalable Multi-Agent 5G Slice Admission

Manoj Prasad Kunasegran, Wai Leong Pang*, Swee King Phang

School of Engineering, Faculty of Innovation and Technology, Taylor's University, Subang Jaya, 47500, Selangor, Malaysia.

*Corresponding author(s). E-mail(s): waileong.pang@taylors.edu.my;

Abstract

Fifth generation (5G) network slicing must admit heterogeneous requests across cooperating base stations under per-slice guarantees. Multi-agent learning can adapt to changing load, yet learned controllers are rarely tested against trivial policies or on independent seeds. We present AXIOM, short for Adaptive Cross-cell Integrated Orchestration of Multi-slice admission. Its encoder, shared by every gNodeB (gNB), feeds per-slice decision heads, keeping its size fixed as clusters grow. A pre-registered protocol with both checks compares it with single-agent and independent deep Q-networks (DQN), multi-agent proximal policy optimisation (MAPPO), value decomposition, and tuned rules, under static and coupled time-varying load. At 7, 19, and 37 gNBs, AXIOM keeps Ultra-Reliable Low-Latency Communication (URLLC) and Enhanced Mobile Broadband (eMBB) service-level compliance above 93%, even when trained only at 7 gNBs, while every learned baseline stays below 84%. A static rule shedding all machine-type requests keeps at least 99.97%, and a tuned rule beats every learner at its tuning conditions. When load rises 10% to 20% past them, AXIOM overtakes that rule by up to 8.7 reward per step. At 37 gNBs, AXIOM leads the best learned baseline by 36.3 to 36.5 points, with a URLLC backlog delay 7 to 44 times shorter than the learned baselines and 7,782 parameters against up to 682,058. Under coupling, AXIOM earns 26% to 36% more reward than MAPPO at 3 and 19 gNBs, and 42% more than that static rule at 3 gNBs. AXIOM is therefore viable for scalable 5G slice admission.

Keywords: Network Slicing, Slice Admission Control, Multi-Agent Reinforcement Learning, Graph Neural Networks, Open RAN

M. P. Kunasegran, orcid.org/0009-0005-3169-1873. W. L. Pang, orcid.org/0000-0001-8407-5648.
S. K. Phang, orcid.org/0000-0002-7877-8766.

1 Introduction

Network slicing virtualises the Radio Access Network (RAN) to serve Ultra-Reliable Low-Latency Communication (URLLC), Enhanced Mobile Broadband (eMBB), and Massive Machine-Type Communication (mMTC), each under its own Service Level Agreement (SLA). Our earlier studies showed that deep reinforcement learning, including a deep Q-network (DQN), can protect URLLC SLAs at a single congested gNodeB (gNB) [22], balance load across a gNB cluster [23], and run live on a real gNB [25]. However, those controllers used a centralised approach, whereas a real deployment spans many gNBs that share spectrum planning and a cluster-wide load-balance objective. Consequently, admission control across such a cluster becomes a multi-agent problem. In this research, we ask whether per-gNB agents that share a view of the whole cluster can decide better than a single centralised agent or independent per-gNB agents. Fig. 1 places our research question within the broader system context.

This paper introduces AXIOM, an acronym for Adaptive Cross-cell Integrated Orchestration of Multi-slice admission. It is designed for the Open Radio Access Network (O-RAN), a softwarised RAN architecture with open, standardised interfaces between its components. One encoder, shared by every gNB, feeds a separate Q-head for each slice, under centralised training and decentralised execution. The encoder is a graph attention network (GAT) [45], and the per-slice heads, each a small multi-layer perceptron (MLP), carry AXIOM's advantage as the cluster grows. Attention itself adds nothing measurable, as Section 6.3 shows. Three gaps found in our prior systematic review of 67 core studies motivate pairing AXIOM with a stricter evaluation, as Section 2 details. No reviewed study checks whether its outcome metric can score a degenerate policy, such as one that accepts every request, as well as a correct one, which blocks just enough low-priority requests to protect the priority slices. In addition, none replicates a headline finding on an independent seed sample, and none tests whether a live testbed reaches the contention that its offline claims assume. A seed fixes the random draws of one run, such as its starting weights and the traffic it trains and is tested on. A finding that also holds on seeds never used before is therefore unlikely to be an artefact of one sample.

We evaluate AXIOM in two offline simulators of a gNB cluster. Firstly, we consider a simple environment (Env A) with a static load, where shedding every mMTC request is close to optimal. Secondly, we consider Env B, which adds time-varying load and inter-cell coupling, so shedding one fixed slice is no longer optimal. We then compare AXIOM with a single-agent deep Q-network (DQN-SA), independent per-gNB deep Q-networks (DQN-I), multi-agent proximal policy optimisation (MAPPO), and tuned threshold rules. We also grow the cluster to 7, 19, and 37 gNBs under three layouts. Further campaigns test federated averaging (FedAvg), differential privacy, and network disruption. The contributions of this paper are as follows.

1. AXIOM, a slice-admission controller with a shared cluster encoder and decentralised per-slice Q-heads, whose parameter count does not grow with the cluster

and whose per-slice heads, rather than attention, carry its advantage at scale. A policy trained at seven gNBs runs unchanged at 19 and 37 gNBs and keeps above 95% priority-slice compliance there in Env A.

2. Env A, a shared-budget environment with static load, built after a trivial-policy check showed that accepting every request beat every learned policy in our original simulator. In Env A, AXIOM keeps priority-slice compliance above 93% at 7, 19, and 37 gNBs, while both DQN baselines and MAPPO stay below 84%.
3. Env B, a more demanding environment with time-varying load and inter-cell coupling. There AXIOM leads DQN-SA, DQN-I, and MAPPO at three and nineteen gNBs and beats a static shedding rule at three, while a tuned threshold rule sets a bar that no learned policy reaches under its tuning conditions.
4. An evaluation protocol for learned slice admission control that combines trivial-policy checks, per-slice correctness metrics, independent-seed replication, tuned rule baselines, and a pre-registered analysis. It applies established practice for evaluating reinforcement learning [2, 17] to slice admission and adds trivial-policy checks that this domain needs.

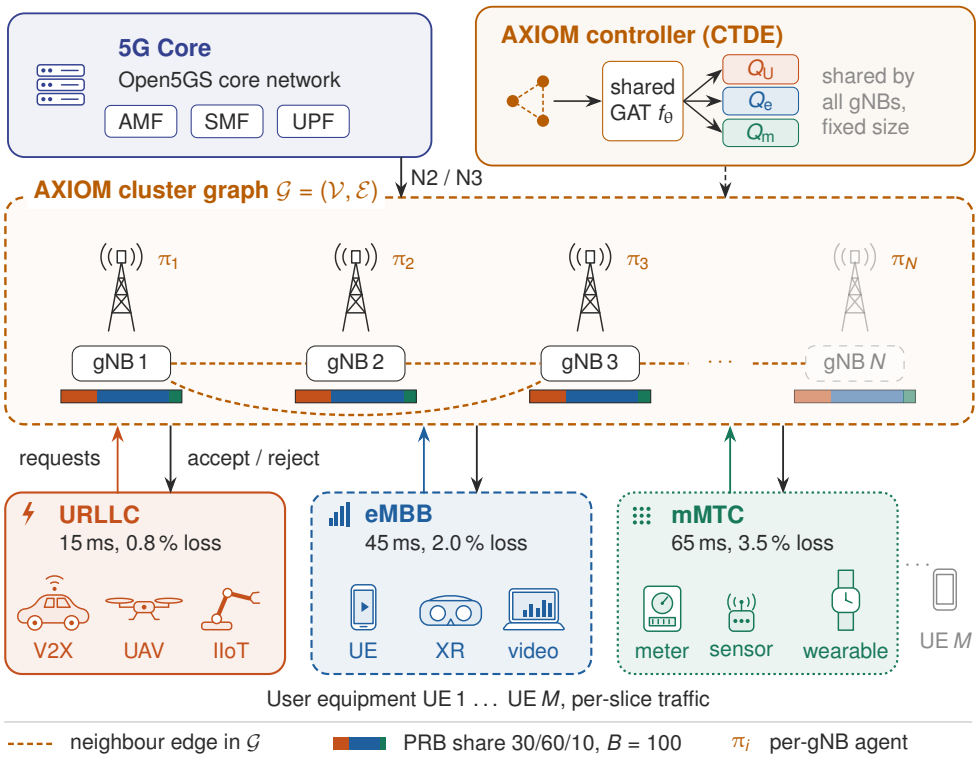


Fig. 1 Conceptual system view. A cluster of N gNBs, coordinated by the shared AXIOM encoder and backhauled to a 5G core, serves M user equipment (UE) devices across three slices with the latency and loss budgets of Table 2. Each gNB hosts an admission agent π_i over a budget of $B = 100$ PRBs. AXIOM is evaluated offline, and the live rig runs one gNB

The remainder of this paper is organised as follows. Section 2 reviews related work, Section 3 formulates the admission problem, and Section 4 presents the AXIOM architecture. Section 5 describes the experimental setup and evaluation safeguards, and Section 6 reports the results. Finally, Section 7 discusses the limitations, and Section 8 concludes the paper.

2 Related Work

Our prior systematic review [24] screened 6,286 records down to 67 core studies from 2024 to 2026 on machine-learning-driven, user-centric distributed network slicing. The review organised them around three enabling pillars, namely topology-aware graph representation, distributed multi-agent control, and privacy-preserving federation. Each pillar is well developed on its own, but they are rarely combined under realistic constraints, and never jointly validated in a live deployment. Our recently presented conference paper [25], due to be published in IEEE Xplore, validated the single-agent case live, and this paper combines all three pillars offline on clusters of up to 37 gNBs.

Studies from the review's core sample show the state of each pillar. Learned admission control uses sequential twin actor-critics [31] and digital-twin-accelerated deep reinforcement learning [41]. Multi-agent control spans sample-efficient multi-agent DQNs for inter-slice orchestration [12], scalable frameworks for dynamic RAN slicing [27], and cooperative agents for energy-efficient slicing [32]. A graph neural network with meta-learning makes bandwidth allocation scalable and transferable [16], and Ahmadi et al. [3] study topology generalisation through attention. Federated deep reinforcement learning has been applied to splitting virtual network functions [5] and to slice mobility [29]. Two core studies combine two of the three pillars, graph attention with multi-agent learning for task offloading [30] and a graph neural network with multi-agent learning for resource orchestration [9]. Closest to our evaluation concern, Tsampazi et al. [44] compare many deep reinforcement learning agents for O-RAN in an automated pipeline, a comparison that our trivial-policy and independent-seed checks extend. None of these core studies checks its metric against a trivial policy or replicates a finding on independent seeds.

Apart from the core studies above, the works cited in the rest of this section lie outside the review's 67 core studies. They add earlier foundations, studies published in this journal, and further recent work. AXIOM also sits inside the broader O-RAN effort to run learned control loops over standard interfaces, which Polese et al. [34] survey and which ColO-RAN [33] and Tsampazi et al. [43] exercise on large experimental platforms. Learned slice admission predates O-RAN, from market-driven admission [8] to reinforcement learning for RAN slicing [36], and Khani et al. [19] survey it for cloud RAN. OACR^2 [26] uses request forecasting to maximise the infrastructure provider's long-term revenue. AXIOM instead protects priority-slice SLAs, the objective of an operator bound by strict per-slice guarantees. Cooperative agents usually train by value factorisation such as QMIX [35] or with a centralised critic as in MAPPO [47], and constrained multi-agent learning has been applied to O-RAN slicing at scale [48]. Recent work in this journal applies actor-critic learning [21] and

Table 1 Closest prior work

Work	Problem	Multi-agent	Graph encoder	Federated	Max cells	Size-invariant params	Zero-shot size transfer	Rule baselines
Shao et al. [37]	Slicing resource management in dense cells	✓	✓[a]	n.r.	19	n.r.	n.r.	✓
Sulaiman et al. [39]	Joint slicing and slice admission in cloud RAN	✓	n.r.	n.r.	n.r.	n.r.	n.r.	✓
Ahmadi et al. [3]	RAN and edge-computing slicing with admission control	✓	✓[a]	n.r.	n.r.	n.r.	n.r.	n.r.
Zangooei et al. [48]	SLA-constrained RAN slicing in O-RAN	✓	n.r.	n.r.	1	◐[e]	◐[e]	✗
Mohajer et al. [30]	SLA-aware task offloading and service orchestration	✓	✓[a]	n.r.	n.r.	n.r.	n.r.	n.r.
Bouroudi et al. [9]	Energy-efficient network-function placement	✓	✓[b]	n.r.	n.r.	n.r.	n.r.	n.r.
AXIOM	Per-slice admission across a gNB cluster	✓	✓[c]	✓[d]	37	✓	✓	✓

✓ reported, ◐ partly, ✗ not used, n.r. not reported in the work. [a]Attention. [b]Graph neural network. [c]Shared GAT or MLP. [d]FedAvg variant. [e]Over the number of slices in a single cell, not over cells.

double deep Q-networks [18] to related problems. Other studies in it address delay-aware allocation [4], federated learning [20], and multi-agent relay selection [38]. None of these studies checks its metric against a trivial policy or replicates on independent seeds. Such checks have overturned published results in other areas of networking. In optical networks, Doherty et al. [13] recreated five landmark reinforcement learning studies of resource allocation and found that simple tuned heuristics outperformed the published learned allocators. Our trivial-policy and tuned-rule checks bring the same scrutiny to slice admission. For slice admission itself, Asdikian et al. [6] attribute a learned agent's decisions to its input features with Shapley values, so that operators can verify its behaviour before deployment. Our checks on the metric complement this view of the policy. Unlike the learned controllers above, AXIOM shares one encoder across agents and gives every slice its own decision head, which Section 6.3 shows matters as the cluster grows.

Two further works come close on a single axis of this problem. Shao et al. [37] show that graph attention strengthens inter-agent cooperation. Sulaiman et al. [39] are the closest prior art to joint slicing and admission control, yet neither study checks for degenerate policies or replicates on independent seeds. Table 1 sets AXIOM beside the six closest works from both groups, and none of them pairs per-slice admission with a size-independent shared controller and evaluates it against trivial and tuned rules on independent seeds.

Robustness studies [14, 42] test disruptive conditions, as our disruption campaign does, yet neither checks whether its robustness metric is trustworthy or replicates a finding. Zhang et al. [49] and Yasin et al. [46] are the closest precedents for periodic synchronisation and for federated learning with differential privacy, but both secure the controller rather than characterise a control policy's own privacy-utility trade-off. Gong et al. [15] combine federated training with multi-agent control, two of the three

pillars. Like every work above, each of these evaluates its own outcome metric once, on one seed or run.

3 System Model and Problem Formulation

This section formulates slice admission across a gNB cluster as a decentralised decision problem and describes the two simulators that evaluate it. We consider a cluster of N gNBs, from three in the main campaigns to 37 in the scaling study. Each gNB serves URLLC, eMBB, and mMTC under a shared per-gNB Physical Resource Block (PRB) budget B. At each control step every gNB holds zero or more pending requests, each tagged with a slice, and its agent accepts or rejects each one, conditioned on the cluster state. Each decision moves the slice's PRB-ratio ceiling, the control that the ORANSlice fork of OpenAirInterface (OAI) exposes to xApps, the external control applications of O-RAN. The binary action matches our single-gNB work, so any coordination benefit comes from the architecture. At three gNBs the cluster is treated as fully connected, a modelling choice, and the scaling study adds ring and hex-grid layouts. The model works at the PRB level and abstracts the radio channel. Channel quality changes how many PRBs a slice needs for the same traffic, and the model folds this into the per-step demand noise. A channel-aware extension that maps the signal-to-interference-plus-noise ratio to PRB demand is left to future work.

The step reward follows our earlier formulations [22, 23]. It combines a per-slice service term net of SLA-violation penalties with a congestion charge on admitted volume and a load-imbalance penalty. For slice k, ω_k is the priority weight and R_k the per-accept reward. Its admitted count is n_k and its violation penalty λ_k, and $\mathbf{1}[v_k]$ flags an SLA violation in that step. With these terms, the reward of step t is then

$$r_t = \sum_k \big(\omega_k R_k\, n_k - \lambda_k\, \mathbf{1}[v_k]\big) \;-\; \kappa\, C_t \|\mathbf{a}_t\|_1 \;-\; \beta\,(1 - F_t), \tag{1}$$

where $\|\mathbf{a}_t\|_1$ is the total admitted this step and C_t the cluster-mean congestion level. The congestion coefficient is κ, and $F_t \in [0, 1]$ is the min-over-max PRB-utilisation fairness ratio with load-balance weight β. The violation flag v_k follows from a continuous SLA margin m_k. The correctness-aware metrics of Section 5 also build on this margin. We take it as the worse of the queue and loss channels,

$$m_k = \min\Big(1 - \tilde{q}_k,\; 1 - \tfrac{\ell_k}{\ell_k^{\max}}\Big), \qquad v_k \Leftrightarrow m_k \le 0, \tag{2}$$

with $\tilde{q}_k$ the slice backlog relative to its budget, and ℓ_k and $\ell_k^{\max}$ its loss proxy and loss budget. The weights rank the slices by the cost of a missed SLA. URLLC carries time-critical traffic with the tightest budgets, 15 ms of latency and 0.8% loss, so it has the largest weight and penalty and is never the slice to shed. eMBB carries throughput-heavy traffic with looser budgets, and mMTC carries small, delay-tolerant device reports with the loosest budgets and the smallest weight. Under congestion the intended policy is therefore differentiated shedding, which sheds mMTC first, sheds eMBB only when URLLC's queue builds, and keeps accepting URLLC.

Table 2 Simulation and training parameters not stated in the text, shared by both environments

Slice parameter	URLLC	eMBB	mMTC
Floor and cap, PRB ratio	5 to 33	10 to 66	5 to 11
Priority weight ω_k	12.0	3.5	0.3
Violation penalty λ_k	10.0	5.0	2.5
Per-accept reward R_k	1.0	1.0	1.0
Latency budget (ms)	15	45	65
Loss budget (%)	0.8	2.0	3.5
Simulator			
Congestion κ, balance β	1.5, 1.0		
Episode	60 steps of 5 s, queues emptied at start		
Requests per step	N, uniform slice and gNB, at most $4N$ pending		
Demand noise, gate step	$0.005B$ per step, 5 PRB-ratio points per decision		
Learning			
Value-based learners	Adam, learning rate 10^{-3}, ϵ from 1.0 to 0.05 at $\times 0.985$ per episode, replay 10,000, minibatch 16, warm-up 32, target sync every 10 episodes		
AXIOM network	Two-layer, four-head GAT, hidden and output 16, raw-feature skip, per-slice Q-heads of hidden 32, Huber loss, gradient clip 1.0		
DQN baselines	Two hidden layers of 128, squared-error loss		
MAPPO	Learning rate 10^{-3}, 10 epochs, minibatch 64, clip 0.2, entropy 0.01, advantage $\lambda = 0.95$		

Admission is therefore a decentralised partially observable Markov decision process. At step t the state s_t comprises every gNB's queues, offered demand, and slice ceilings. Agent i observes its own feature row $\mathbf{x}_i$, which the shared encoder combines with the other rows of $\mathbf{X}_t$. For each pending request it chooses $a \in \{\text{accept}, \text{reject}\}$, the shared-budget dynamics move the cluster to s_{t+1}, and every agent receives the common reward r_t of Eq. (1). The agents jointly seek a policy π that maximises the discounted return over an episode of T=60 steps,

$$\max_{\pi} \ \mathbb{E}_{\pi}\Big[\textstyle\sum_{t=0}^{T-1} \gamma^t r_t\Big], \qquad \gamma = 0.95. \tag{3}$$

Each gNB has a budget of B=100 PRBs in both simulators. The nominal shares are 30, 60, and 10 PRBs for URLLC, eMBB, and mMTC. Each slice's mean offered demand is 105% of its share, scaled by a fixed per-gNB multiplier m_i between 0.9 and 1.0. This puts a gNB's mean demand at about 95% to 105% of its budget, before per-step noise and backlog. When admitted demand exceeds B, the slices share it in proportion to demand, so every admitted mMTC request takes resources from URLLC and eMBB. Unserved demand carries forward as backlog within an episode. Each episode starts with empty queues and every slice ceiling at its nominal share. An accept raises a slice's ceiling by 5 PRB-ratio points, up to its cap, and a reject lowers it by 5 points, down to its floor, and relieves one request's demand from the backlog. Each slice then asks for the smaller of its offered demand plus backlog and its ceiling, and the offered PRBs drift 10% per step towards their mean, with noise of $0.005B$. Env B keeps all of this and adds two mechanisms. Each gNB's mean demand follows $0.95\,(1 + 0.3\sin(2\pi(s + \psi_i)/60))$ times its nominal shares, again scaled by m_i, with s the step and ψ_i a phase redrawn per gNB and episode. As a result, demand swings

between about 60% and 124% of the budget. A gNB's usable budget also shrinks with its neighbours' previous load, $B_i(t) = B\,(1 - 0.15\,\bar{u}_{\mathcal{N}(i)}(t-1))$, a first-order model of inter-cell interference over the same adjacency graph the encoder reads. In Env B shedding every mMTC request leaves URLLC compliance at 61.5% at three gNBs, whereas a rule that also sheds eMBB when the URLLC queue builds keeps it at 98.7%. Per-slice compliance counts a slice as compliant only when every gNB meets it, which saturates in Env B at nineteen gNBs, so there we also report per-gNB compliance. Fig. 2 traces one control step of both simulators, and Table 2 lists the remaining simulation and training parameters.

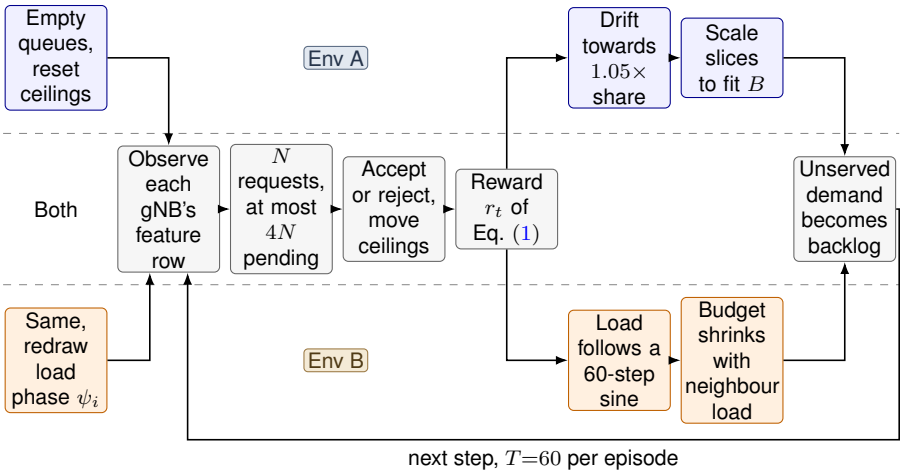


Fig. 2 One control step of both simulators, repeated for T=60 steps per episode. The middle lane holds the steps both share, and the outer lanes the steps where Env A and Env B differ. The text gives each step in full

4 The AXIOM Architecture

This section presents the AXIOM architecture, its training rule, and its federated variant. AXIOM differs from its DQN baselines in what each policy sees and shares. AXIOM trains one encoder over the whole cluster from every agent's experience and reads out one Q-head per slice. DQN-SA is one controller over a flattened joint state, and DQN-I gives each gNB its own learner over local features only. The two baselines are therefore ablations that remove the shared encoder.

4.1 Shared Encoder, Decentralised Q-Heads

Fig. 3 shows the AXIOM architecture and its data flow. At every step each gNB forms a feature row $\mathbf{x}_i$ of its PRB-used ratio, congestion level, and normalised queue length, and the rows stack into $\mathbf{X} \in \mathbb{R}^{N\times d}$. A two-layer, four-head GAT over the adjacency graph maps $\mathbf{X}$ to one normalised embedding $\mathbf{z}_i$ per gNB, and Section S4 of Online Resource 1 gives its attention equations. Every agent's loss updates this encoder, which

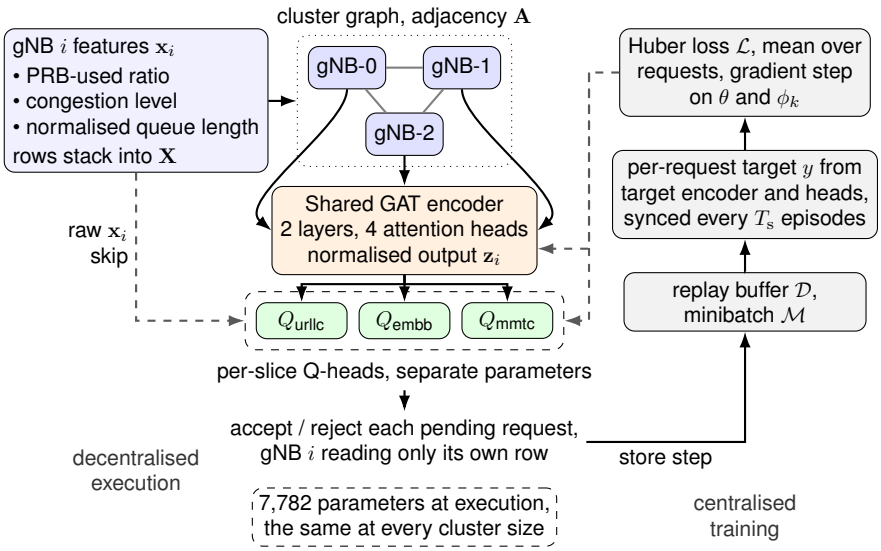


Fig. 3 AXIOM architecture. A shared graph-attention encoder produces one embedding per gNB, and per-slice Q-heads read that gNB's embedding and raw features. Training on every agent's requests is centralised, and each gNB acts on its own row

Algorithm 1 Centralised training and decentralised execution of AXIOM

Require: encoder f_θ, per-slice Q-heads $\{Q_{\phi_k}\}$, targets $f_{\theta^-}, \{Q_{\phi_k^-}\}$, buffer $\mathcal{D}$, adjacency $\mathbf{A}$, discount γ, sync period T_s

for each step t, observing joint node features $\mathbf{X}_t$ **do**
  $\mathbf{E}_t \leftarrow f_\theta(\mathbf{X}_t, \mathbf{A})$ {centralised, one row per gNB}
  for all pending requests at gNB i, slice k **do**
    $a \leftarrow \arg\max_a Q_{\phi_k}([\mathbf{E}_t[i], \mathbf{X}_t[i]])[a]$, ϵ-greedy {decentralised, own row only}
  end for
  step env, store $(\mathbf{X}_t, \mathbf{X}_{t+1}, \{(i,k,a)\}, r_t, \eta_t)$ in $\mathcal{D}$
  if $|\mathcal{D}| \geq$ warm-up **then**
    sample minibatch $\mathcal{M}$, then $\mathbf{E}_\mathcal{M} \leftarrow f_\theta(\mathbf{X}_\mathcal{M}, \mathbf{A})$, $\mathbf{E}'_\mathcal{M} \leftarrow f_{\theta^-}(\mathbf{X}'_\mathcal{M}, \mathbf{A})$
    per request, $y \leftarrow r + \gamma(1-\eta)\max_{a'} Q_{\phi_k^-}([\mathbf{E}'_\mathcal{M}[i], \mathbf{X}'_\mathcal{M}[i]])[a']$
    $\mathcal{L} \leftarrow$ mean per-request Huber loss, never summed per step, then update $\theta, \{\phi_k\}$ with gradient norm clipped at 1.0
  end if
  every T_s episodes, $\theta^- \leftarrow \theta$, $\phi_k^- \leftarrow \phi_k$
end for

is centralised training, while each agent decides its own pending requests from its own row $\mathbf{z}_i$, which is decentralised execution.

Two design choices in AXIOM matter most at scale. One small MLP per slice gives each slice its own parameters, so URLLC's large priority weight cannot swamp mMTC's smaller learning signal in a shared head, which Section 6.3 confirms at nineteen gNBs. Each node's raw features are also concatenated onto its normalised embedding before the heads, $Q_{\phi_k}([\mathbf{z}_i, \mathbf{x}_i])$, so a head sees absolute queue and congestion levels. From here on, $\mathbf{z}_i$ denotes that concatenated input to the heads. This skip connection and a learning rate of 10^{-3}, both chosen on Env B pilot seeds before any reported run, are the only changes from the original AXIOM design.

In training, every pending request at gNB i, slice k is its own temporal-difference sample, and Algorithm 1 gives the full training loop. For a request with immediate reward r and next-state embedding $\mathbf{z}'_i$, the target y at discount γ and the minibatch loss are

$$y = r + \gamma\,(1-\eta)\max_{a'} Q_{\phi_k^-}\big(\mathbf{z}'_i\big)[a'], \qquad \mathcal{L} = \frac{1}{|\mathcal{M}|}\sum_{(i,k,a)\in\mathcal{M}} \ell_\delta\Big(Q_{\phi_k}(\mathbf{z}_i)[a] - y\Big), \tag{4}$$

where $Q_{\phi_k^-}$ is slice k's target Q-head, $\eta \in \{0,1\}$ the flag that marks an episode's last step, $\mathcal{M}$ the minibatch of requests, and ℓ_δ the Huber loss. Section 6.3 shows that a summed per-step target works as well, so the per-request scheme is a simplification rather than a source of AXIOM's advantage.

4.2 Federated Training with Differential Privacy

Section S5 of Online Resource 1 shows the federated variant, in which each gNB holds a private copy of the network, trains only on its own requests, and every N_{loc} local steps sends its weights for FedAvg [28]. Each client can add differential privacy through a differentially private stochastic gradient descent (DP-SGD) step [1], which clips its minibatch gradient and adds Gaussian noise of multiplier σ. The privacy loss is accounted for with zero-concentrated differential privacy [10]. Section S4 of Online Resource 1 gives the clipping, the privacy accounting, and its assumptions.

5 Experimental Setup

This section describes the campaigns, the baselines, and the statistical tests, followed by the safeguards that protect every reported result. The protocol follows established practice for evaluating reinforcement learning [2, 17], with many seeds, interval estimates, and paired tests, and adds trivial-policy checks specific to slice admission. Every campaign uses 300 training and 50 evaluation episodes per seed, where a seed fixes one network initialisation and one environment stream, and each seed is evaluated on a held-out stream. The three-gNB campaigns use 60 seeds, 900 to 929 and 11000 to 11029, with a disjoint replication sample, 1000 to 1009. The Env A scaling campaigns use 30 seeds per layout at seven gNBs, 60 at nineteen, and at thirty-seven 60 on the hex layout and 30 on each other layout. Half of each of these samples, the seeds from 11000, was added by a later pre-registered extension with unchanged

settings, and every result for these cells uses all their seeds. The Env B scaling campaigns use 15 seeds per layout. MAPPO [47] is a published multi-agent baseline with a shared actor and a centralised critic, trained on-policy for the same 300 episodes on the same reward. Its learning rate, epochs, and minibatch size were tuned on pilot seeds, because untuned it collapsed in four of five pilot seeds.

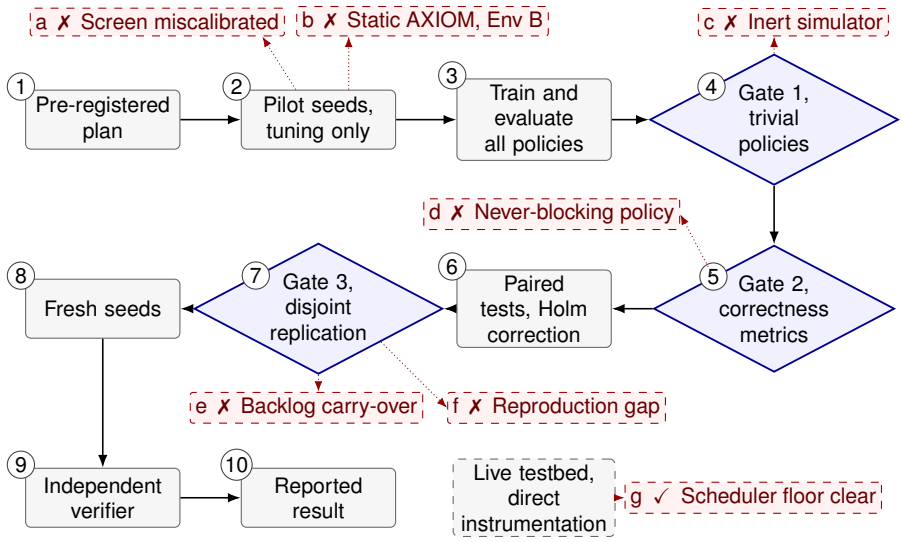


Fig. 4 Evaluation protocol in ten steps, with the three gates as diamonds. Dashed callouts name what each check found. a, A miscalibrated validation screen led to an amended plan, back to step 1. b, AXIOM stayed static in Env B, which led to faster learning, back to step 3. c, An inert simulator led to the shared-budget rerun, back to step 3. d, A never-blocking policy is counted as collapsed. e, Backlog carried across episodes led to an episode reset and a rerun, back to step 3. f, A reproduction gap led to a retracted claim. g, The live testbed found no scheduler floor, over more than 100,000 readings

Rules need no learning, and accept-all simply admits every request. A threshold rule sheds mMTC whenever its gNB's congestion level reaches a threshold τ, chosen in Env A on training streams only. Thresholds of 0, 0.5, and 0.8 tie exactly on the training streams, and we keep the lowest, τ=0. The rule then sheds every mMTC request, and we call it the static shed-mMTC rule, or static rule for short. Shedding mMTC is the natural first step, since it has the smallest weight and the loosest budgets. In Env A, where demand runs 5% above the nominal shares, removing it frees enough PRBs to keep URLLC and eMBB compliance above 99.98% on average, as Section 6.1 shows. It is the best rule of its family on training streams. It is a strong reference rather than a proven optimum, since across our Env A samples learned policies exceeded it in 33 of 540 seed-level evaluations, by at most 0.40 reward per step. In Env B we add two tuned rule families, each chosen on training streams over a grid of thresholds and evaluated on the held-out streams. Operators commonly protect slices with thresholds of this kind, so a learned controller earns its place only by beating the best threshold an engineer could set, and the tuned rules give the learners that bar. The local family

sheds mMTC on the gNB's own congestion or priority-slice queue and sheds eMBB on its URLLC queue, while the neighbour-aware family also reads the mean congestion of the gNB's neighbours. eMBB is the second slice to shed. Under Env B's load peaks, shedding mMTC alone cannot protect URLLC, as Section 6.4 shows, and eMBB frees the most room, since it has the largest PRB ceiling and the next-smallest weight. Two further runs repeat the DQN baselines at a learning rate of 10^{-4}.

All learned policies share $\gamma = 0.95$, the training budget, and the exploration schedule of Table 2, and Section S4 of Online Resource 1 gives the design of the sensitivity studies.

All confidence intervals (CIs) are 95% bootstrap percentile intervals over 10,000 resamples. Paired comparisons use the Wilcoxon signed-rank test, and each family of comparisons against AXIOM is Holm-corrected for multiple testing. The test drops exactly tied pairs, so where two policies make identical decisions on many seeds, as AXIOM and the static rule do in Env A, a comparison rests on the remaining seeds.

The settings, seeds, and four hypotheses of the final campaign were fixed in a pre-registration committed to the project's version-control history before any of its runs, rather than lodged with an external registry. Every later campaign was pre-registered the same way. The Env A baselines that this campaign reuses come from an earlier rerun whose plan was committed one minute after that rerun was launched and before any of its results were read. Section S4 of Online Resource 1 lists the reused results. H1 is that AXIOM beats both trivial rules in Env B, and H2 compares it with every learned baseline and tuned rule in Env B. H3 asks whether graph attention beats a per-node MLP encoder in Env B at seven gNBs, and H4 compares AXIOM with every baseline at nineteen gNBs in Env A.

A pre-registered validation screen checks every trained checkpoint on its own training stream before evaluation. Every seed is reported whether or not it passes, and Section S4 of Online Resource 1 gives the screen's criteria.

Three safeguards protect the results, and Fig. 4 places them in the evaluation protocol with the failure each one caught. A trivial-policy check evaluates accept-all and one-slice shedding before any learned policy is trusted, and it exposed an earlier simulator in which accepting every request was optimal.

Correctness-aware metrics form the second safeguard of the evaluation. Block precision $\pi = b_{\mathrm{mMTC}}/b$ is the share of a policy's b blocked requests that target mMTC, and a seed that never blocks counts as a collapse rather than a success. Per-slice SLA compliance then shows whether shedding protects URLLC and eMBB, whereas all-slice compliance scores a correct policy and a failed one alike, because correct shedding gives up mMTC's SLA by design.

Finally, every headline comparison is recomputed on an independent, disjoint seed sample. This caught queue backlog carried over between episodes, and every result here comes from the corrected simulator, which empties all queues at the start of each episode.

6 Results

We summarise per-slice compliance as priority-slice compliance, the mean of URLLC and eMBB compliance, and call a trained policy collapsed when it accepts every request. Fig. 5 sets out how the four learned policies stand as the cluster grows. Throughout, a reward margin is a difference in mean reward per step, a compliance margin is in percentage points, and a load level is a percentage of the nominal offered demand.

6.1 Static Load, Three gNBs

In the sixty-seed sample of Table 3, AXIOM never collapses and comes closer to the static rule's reward than any other learner. AXIOM earns 1.061 more reward per step than DQN-I, with 95% CI [0.77, 1.38], and 2.482 more than MAPPO, with 95% CI [2.05, 2.94], at Holm-adjusted $p < 0.0001$. It earns 0.312 more than DQN-SA, with 95% CI [0.10, 0.55], at $p = 0.0021$. This last margin is new with the added seeds. On the first 30 seeds it was 0.311 and missed correction, at $p = 0.066$, and the 30 added seeds alone give 0.312 at $p = 0.051$. The static rule earns 0.101 more than AXIOM, with 95% CI [0.02, 0.21], a small gap that the added seeds make significant, at $p = 0.040$. A disjoint replication on seeds 1000 to 1009, pooled with the first 30 seeds, favours AXIOM more strongly, as Section S5 of Online Resource 1 reports, but fresh seeds in Section 6.6 do not reproduce the lead over DQN-SA.

Table 3 Env A, three gNBs, 60 seeds per policy, 95% bootstrap CIs. Rules above the line, learned policies below it. W/T/L counts the seeds where AXIOM earns more, the same, or less. Precision averages the seeds that block at all

							Compliance (%)		
Policy	Reward per step	95% CI	Of static rule (%)	AXIOM W/T/L ($n = 60$)	Precision	Collapsed	URLLC	eMBB	mMTC
Static rule	12.849	[12.80, 12.90]	100.0	5/40/15	1.000	0/60	100.0	100.0	36.5
AXIOM	12.748	[12.63, 12.85]	99.2	–	0.996	0/60	99.9	98.2	40.2
DQN-SA	12.437	[12.20, 12.64]	96.8	31/16/13	0.990	1/60	99.5	94.7	42.6
DQN-I	11.687	[11.39, 11.95]	91.0	56/0/4	0.972	0/60	99.2	88.9	40.4
MAPPO	10.266	[9.79, 10.72]	79.9	58/1/1	0.728	3/60	97.9	61.6	66.5

6.2 Cluster-Size Scaling

We grow the Env A cluster from three gNBs to seven and nineteen under fully-connected, ring, and hex-grid layouts. A pre-registered extension then adds a thirty-seven-gNB cluster on the same layouts. We test the per-layout gaps on reward and on priority-slice compliance, Holm-correcting the 18 comparisons at each size. Fig. 5 traces the result across sizes, and Section S6 of Online Resource 1 gives the per-slice detail.

At seven gNBs, under every layout, AXIOM keeps 15.1 to 29.8 percentage points more priority-slice compliance than each baseline and earns 3.38 to 7.82 more reward

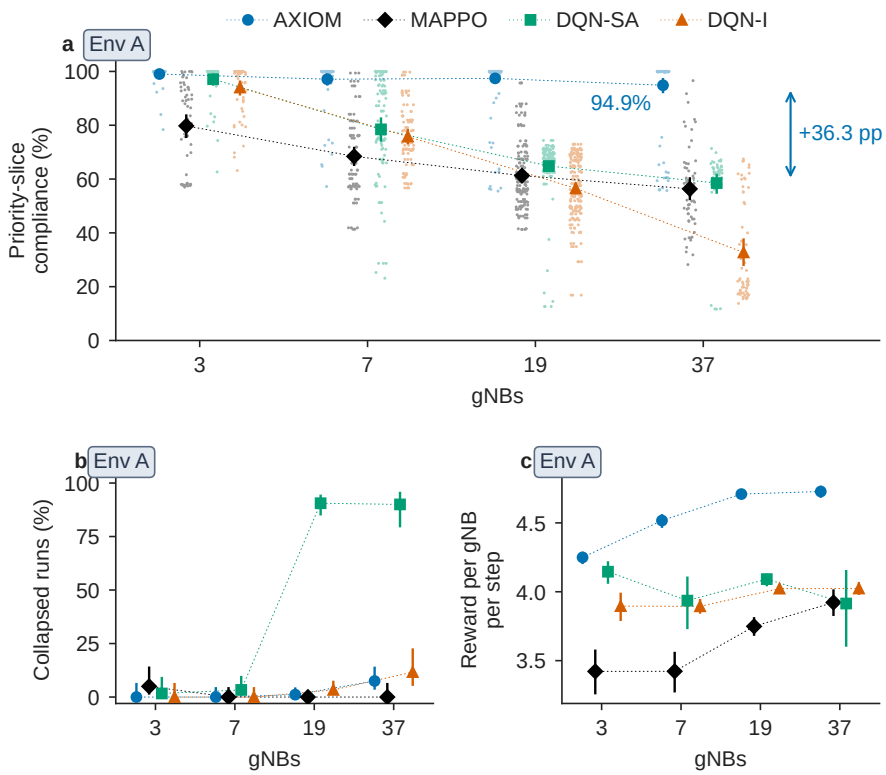


Fig. 5 Env A as the cluster grows, all layouts pooled, with the layout-independent baselines run once per seed at 37 gNBs. a, Priority-slice compliance, one dot per run, with the mean and 95% CI. b, Share of trained policies that collapse into accept-all, with Wilson 95% intervals. c, Reward per gNB per step, mean and 95% CI. Dotted lines join each policy's means across sizes

per step. All 18 comparisons hold after Holm correction, at adjusted $p \leq 0.0004$. At nineteen gNBs the gaps widen, and they answer hypothesis H4. AXIOM keeps a pooled priority-slice compliance of 97.4% with 95% CI [96.0%, 98.7%]. Under every layout AXIOM keeps 32.1 to 41.4 percentage points more compliance than every baseline and earns 11.4 to 18.4 more reward per step, winning 56 to 60 of 60 seeds. All 18 comparisons hold after Holm correction, at adjusted $p \leq 0.0001$.

At thirty-seven gNBs the lead grows further, and AXIOM keeps a pooled priority-slice compliance of 94.9% with 95% CI [92.4%, 97.1%]. Under every layout AXIOM keeps 32.3 to 66.5 percentage points more compliance than every baseline and earns 23.0 to 30.4 more reward per step. Here all 18 comparisons hold after Holm correction within their pre-registered families, at adjusted $p < 0.01$. AXIOM collapses more often than at nineteen gNBs, as Fig. 5b shows, but even its collapsed runs keep 57% to 65%, the level of the best baselines. A pre-registered replication on 20 fresh seeds confirms the result. There AXIOM keeps 95.1% priority-slice compliance against 34.1% to 58.5% for the baselines and leads each on compliance and on reward at Holm-adjusted $p < 0.001$. It collapses in 1 of the 20 seeds, against 18 for DQN-SA. On these

clusters AXIOM serves 99.8% of the offered URLLC demand and 99.0% of the eMBB demand, against 94.2% to 97.7% and 85.7% to 97.5% for the learned baselines. Its URLLC delay, the mean backlog over the mean served PRBs in 5 s control periods, is 0.08 with a 95th percentile of 0.42, whereas the baselines reach 0.58 to 3.61 with 95th percentiles of 5.2 to 16.9. Section S8 of Online Resource 1 gives throughput and delay in every setting.

Accepting every request is the natural floor at scale. It keeps 71.2%, 66.8%, and 61.9% priority-slice compliance at seven, nineteen, and thirty-seven gNBs. Every baseline falls below it on average at the two larger sizes. AXIOM stays 26 to 33 points above the floor. Its advantage at scale therefore lies in reliably learning the correct shedding policy, the one the static rule encodes, rather than in coordination beyond that rule. The static rule that sheds every mMTC request keeps 99.97% to 99.99% at every size. A local threshold rule tuned on Env A training streams at thirty-seven gNBs, in a further pre-registered run, settles on this same policy and matches the static rule in all 30 seeds. Where that rule is known, AXIOM comes within 3 points of it up to nineteen gNBs and within 5.1 at thirty-seven, while from seven gNBs up every other learner falls at least 21 points short. Tuning the DQN baselines on pilot seeds does not close the gap, and AXIOM beats each tuned baseline on compliance and on reward at Holm-adjusted $p < 0.001$, by 29.9 to 48.1 points. The published value-decomposition network (VDN) [40] ties AXIOM at three gNBs but trails it by 13.6 points at nineteen, as Section S6 of Online Resource 1 reports.

Fig. 6 shows where the gap comes from, gNB by gNB, in the thirty-seven-gNB hex cluster. A gNB's admission gate for a slice is the PRB-ratio ceiling that its accept and reject decisions move between the slice's floor and cap, averaged over every evaluation step of all 60 seeds. AXIOM keeps the URLLC and eMBB gates open at every gNB,

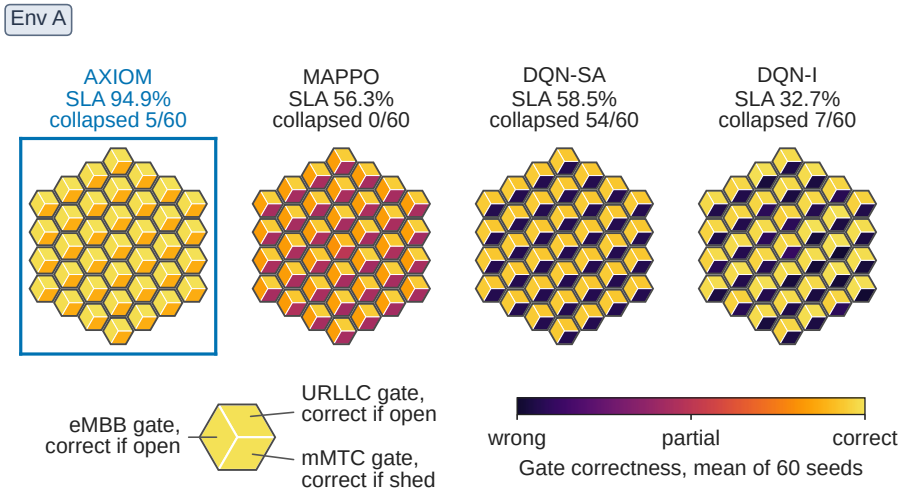


Fig. 6 Decision map of the 37-gNB hex cluster in Env A over 60 seeds. Each hexagon is a gNB at its true position, split into its URLLC, eMBB, and mMTC admission gates, each drawn on the correctness scale below. Headers give each policy's priority-slice compliance on this layout and its collapsed seeds

at 1.00 and 0.99, and closes the mMTC gate to 0.13 even with its five collapsed seeds included. No baseline closes the mMTC gate below 0.55.

AXIOM's encoder and heads have the same shape at every cluster size, so a policy trained at one size can run unchanged at another. A pre-registered run evaluated the seven-gNB checkpoints, never retrained, at nineteen and thirty-seven gNBs on each seed's held-out stream, and Table 4 gives the result on the 15 seven-gNB seeds. In Env A on the hex layout, AXIOM transferred from seven gNBs keeps 96.2% and 95.4% priority-slice compliance at nineteen and thirty-seven gNBs. Accepting every request keeps 65.9% and 61.1%, and MAPPO transferred the same way keeps 64.2% and 61.5%. All four of AXIOM's leads hold at Holm-adjusted $p = 0.00024$. On all three layouts and in Env B at nineteen gNBs, the transferred policy stays within 3.4 points of AXIOM trained at the target size, and no paired difference is significant, with $p \geq 0.068$. On 20 fresh seeds trained for this check, AXIOM moved from seven gNBs keeps 96.2% at nineteen gNBs and 95.2% at thirty-seven. It beats accept-all and MAPPO moved the same way by 27.0 to 33.8 points, at Holm-adjusted $p < 0.001$. The DQN baselines cannot transfer, since DQN-SA reads the joint state of every gNB and DQN-I holds one network for each gNB, with none for the new ones. The Env A result partly reflects how simple the task is there, because the transferred policy earns exactly the size-independent static rule's reward on 9 to 11 of 15 seeds and less on the rest.

Table 4 Zero-shot transfer on the 15 seven-gNB seeds, priority-slice compliance with 95% CIs. Policies from 7 were trained at seven gNBs and run unchanged, and AXIOM at N was trained at the target size. Env B gives per-gNB compliance

	Priority-slice compliance (%)						Rules (%)	
	AXIOM from 7		AXIOM at N		MAPPO from 7			
Env, layout, N	Mean	95% CI	Mean	95% CI	Mean	95% CI	Static	Accept all
A, hex, 19	96.2	[91.6, 99.9]	94.6	[86.9, 100.0]	64.2	[56.6, 72.2]	99.96	65.9
A, hex, 37	95.4	[89.6, 99.9]	94.8	[87.3, 100.0]	61.5	[53.7, 70.2]	99.99	61.1
A, ring, 19	96.9	[91.3, 99.9]	96.8	[92.3, 99.9]	64.2	[56.8, 72.4]	99.96	65.9
A, ring, 37	96.3	[90.3, 99.7]	93.2	[85.3, 100.0]	61.5	[53.6, 70.2]	99.99	61.1
A, fully connected, 19	96.6	[90.6, 100.0]	99.96	[99.9, 100.0]	64.2	[56.8, 72.3]	99.96	65.9
A, fully connected, 37	96.3	[90.1, 100.0]	94.4	[85.9, 100.0]	61.5	[53.8, 70.0]	99.99	61.1
B, hex, 19	79.1	[76.6, 80.8]	77.3	[74.2, 79.8]	72.9	[70.1, 75.6]	79.9	66.4

6.3 Ablations and Sensitivity

Five checks test where AXIOM's behaviour comes from, as Sections S5 and S7 of Online Resource 1 detail. First, the per-slice Q-heads matter as the cluster grows. A variant with one shared head of matched size ties AXIOM at three gNBs. At nineteen gNBs on the ring layout, however, it collapses in 9 of 15 seeds and keeps only 65.9% priority-slice compliance against AXIOM's 96.8% on the same seeds, losing every seed at Holm-adjusted $p < 0.001$. Second, graph attention adds little to AXIOM. In Env A a per-node MLP encoder earns only 0.068 less reward per step than AXIOM at three

gNBs, with 95% CI [−0.06, 0.19], though on 60 seeds the paired test finds this gap significant, at Holm-adjusted $p = 0.015$. In Env B at seven gNBs it keeps 1.12 and 0.95 more reward per step on the ring and hex layouts and wins 10 of 15 seeds in each. These leads in the pre-registered test H3 miss significance after Holm correction, at $p = 0.061$ and $p = 0.095$. On ten fresh ring-layout seeds, however, the MLP encoder led by 1.38 reward per step at $p = 0.006$. Third, a variant with one joint summed target in place of per-request samples is never significantly different. Fourth, the learning rate matters, and AXIOM stays within 0.13 of the static rule's reward under changed load and slice weights. A 24-configuration MAPPO grid and a seeded DQN-SA still lose to AXIOM in every setting tested. Finally, depth-3 decision trees reproduce 86% to 97% of AXIOM's held-out decisions in Env B and shed eMBB once the URLLC queue passes 0.21 at three gNBs and 0.28 at seven after 900 training episodes, near the tuned rule's 0.2. Run as a rule, the trees tie AXIOM at seven gNBs, trail it by 0.19 reward per step at three, and never reach the tuned rule.

6.4 Time-Varying Load with Inter-Cell Coupling

Table 5 and Fig. 7 report Env B at three gNBs, where shedding one fixed slice no longer suffices. AXIOM earns 2.560 more reward per step than shedding every mMTC request and 4.546 more than accept-all. Both gaps hold at Holm-adjusted $p < 0.0001$, as hypothesis H1 predicted. It also leads every learned policy, by 1.121 reward per step over DQN-I, 1.338 over DQN-SA, and 1.801 over MAPPO, and keeps more priority-slice compliance than each, at Holm-adjusted $p < 0.0001$. A tuned rule that also sheds eMBB whenever the URLLC queue passes 0.2 beats every learned policy, earning 0.426 more than AXIOM at $p < 0.0001$. The tuned neighbour-aware rule also outearns AXIOM, by 0.235 per step at $p = 0.012$, a gap that was within chance on the first 30 seeds, while AXIOM keeps 1.2 points more priority-slice compliance than it, at $p = 0.012$.

Table 5 Env B, three gNBs, 60 seeds per policy, 95% bootstrap CIs. Rules above the line, learned policies below it. W/T/L counts the seeds where AXIOM does better, the same, or worse

				W/T/L ($n = 60$)		Compliance (%)		
Policy	Reward per step	95% CI	Of tuned rule (%)	Reward	Priority SLA	URLLC	eMBB	mMTC
Accept all	4.177	[4.01, 4.35]	45.7	60/0/0	60/0/0	43.3	25.4	94.9
Shed all mMTC	6.162	[5.93, 6.40]	67.4	60/0/0	60/0/0	61.5	35.2	36.7
Tuned local rule	9.149	[9.05, 9.25]	100.0	11/0/49	10/0/50	98.7	35.3	38.5
Tuned neighbour-aware rule	8.958	[8.87, 9.04]	97.9	26/0/34	42/0/18	98.7	27.1	89.0
AXIOM	8.723	[8.56, 8.88]	95.3	–	–	95.7	32.6	42.4
DQN-SA	7.385	[7.09, 7.64]	80.7	56/0/4	52/0/8	86.5	26.8	40.5
DQN-I	7.601	[7.34, 7.86]	83.1	56/0/4	53/0/7	89.2	25.9	43.5
MAPPO	6.921	[6.53, 7.30]	75.7	56/0/4	49/0/11	80.0	28.0	55.5

At seven gNBs the tuned rules beat AXIOM in all 15 seeds under every layout. On the ring layout AXIOM keeps less compliance than DQN-SA, at Holm-adjusted $p =$

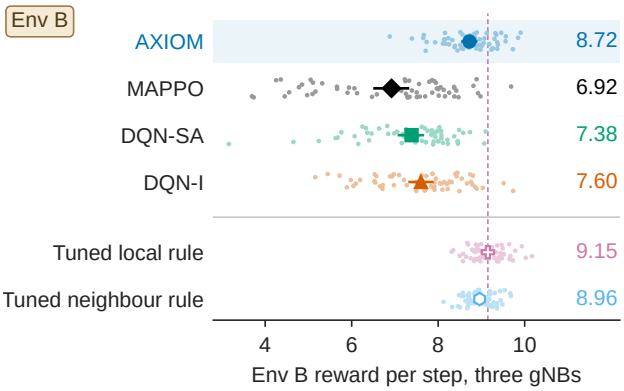


Fig. 7 Env B reward per step at three gNBs, 60 seeds, with per-seed points behind the mean and 95% CI. Learned policies are filled markers and rules hollow. The dashed line marks the tuned local rule's mean

0.012. This gap mainly reflects the 300-episode budget, within which 28 of AXIOM's 45 seeds settle on the static rule's policy and never shed eMBB, which none of its 60 seeds does at three gNBs. With 900 episodes for every learner on the ring layout, AXIOM keeps 55.7% priority-slice compliance against 40.5% to 42.9% for the baselines, at Holm-adjusted $p \leq 0.0012$, though still below the tuned rule's 60.5%. After 900 episodes AXIOM rejects eMBB more often as the URLLC queue grows, much as the tuned rule does. At nineteen gNBs the longer budget does not help, and seven of eight seeds keep the static policy after 900 episodes, as Section S7 of Online Resource 1 reports.

At nineteen gNBs, where cluster-level compliance saturates, AXIOM earns 5.54 to 21.22 more reward per step than every learned baseline and keeps 5.5 to 10.9 percentage points more per-gNB priority-slice compliance, at Holm-adjusted $p \leq 0.013$. The static rule still earns 1.31 more per step, and the tuned local rule carried over from seven gNBs earns 3.81 more and wins every seed. Fig. 8 maps this per-gNB compliance for the four learned policies. Each hexagon is one gNB, shaded from dark at 50% to light at 85% by the mean share of evaluation steps on which it meets the URLLC and eMBB SLAs. The policies only admit or reject requests where they arrive, so the colours show how well each gNB copes with its own offered load. Every policy keeps the least at the centre gNB, whose load the simulator fixes at the nominal level while it draws every other gNB's from 90% to 100% of it. The six inner gNBs have as many neighbours but trail the edge by only 0.5 to 1.8 points. AXIOM keeps more than every learned baseline at each of the nineteen gNBs, by at least 3.8 points, and by at least 4.3 points on the 15 fresh seeds. The carried-over rule leads AXIOM by 13.0 points at the centre and 6.9 points elsewhere, and by 13.2 and 7.0 points on the fresh seeds.

The rule's lead holds only under its tuning conditions. When the load rises to 110% to 120% without re-tuning or retraining, AXIOM wins 21 to 23 of 25 seeds at three gNBs, at Holm-adjusted $p \leq 0.0002$. At nineteen gNBs it earns 6.23, 7.55, and 8.67 more reward per step than that rule, winning 14 of 15 seeds at each level at Holm-adjusted $p < 0.001$. On 15 fresh seeds it leads that rule by 6.84, 8.25, and 9.39 at the

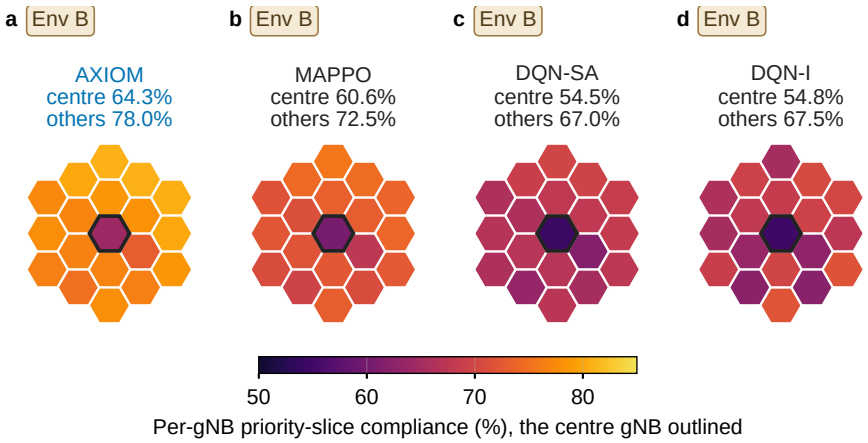


Fig. 8 Per-gNB priority-slice compliance in Env B at nineteen gNBs on the hex layout, mean of 15 seeds. a, AXIOM. b, MAPPO. c, DQN-SA. d, DQN-I. The centre gNB is outlined

same loads, again winning 14 of 15 seeds at Holm-adjusted $p < 0.001$, while the rule leads by 3.33 at nominal load. Past the tuning load AXIOM also out-earns MAPPO, DQN-SA, and DQN-I at seven and nineteen gNBs, at Holm-adjusted $p \leq 0.008$. At three gNBs it leads all three only at 110% load, and from 115% DQN-SA and DQN-I come within 0.21 reward per step of it. A re-tuned rule ties AXIOM up to 115% load and beats it at 120%. Section S7 of Online Resource 1 adds uneven per-gNB load and a per-gNB SLA penalty, under which AXIOM still beats every learned baseline but not the rules.

In Env A, Fig. 5a and c show AXIOM staying between 4.2 and 4.7 reward per gNB per step and above 93% priority-slice compliance, while every baseline loses compliance as the cluster grows. MAPPO is the only learned policy that never collapses at thirty-seven gNBs, where AXIOM collapses in 9 of 120 runs. In Env B every policy loses compliance as the cluster grows, and AXIOM's cluster-level priority-slice compliance falls from 64.1% at three gNBs to 35.1% at seven and 24.7% at nineteen. AXIOM earns the most reward per gNB and keeps the most compliance at three and nineteen gNBs. At seven gNBs it earns 3.36 reward per gNB per step against 3.37 for DQN-SA, which keeps more compliance, 44.3% against 35.1%. That gap reflects the training budget, as shown above. In Env A at nineteen gNBs AXIOM stays within 5.3 points of the static rule as offered load rises from 105% to 120% of the nominal shares, and every baseline stays below AXIOM at each load.

6.5 Model Complexity

Table 6 sets out the cost of each policy. AXIOM's shared encoder keeps its parameter count fixed as the cluster grows, while the DQN baselines grow with it. Its greedy decisions on one CPU thread stay well inside the 5 s control period of the live testbed. Against MAPPO, the cheapest learner, the extra cost buys the most where the environment is dynamic. In Env B AXIOM earns 26% more reward per step than MAPPO

at three gNBs and 36% more at nineteen gNBs. A 24-configuration tuning grid for MAPPO did not close the gap in any setting where it was tested.

6.6 Reproducibility and Fresh-Seed Verification

We checked the results in three ways, with every pass criterion fixed before the run it judged. First, rerunning eleven reported settings in fresh processes reproduced every stored result exactly. Sixteen further reruns, from seven to thirty-seven gNBs in both environments, also matched exactly. The one exception is DQN-SA, which builds its network before setting the seed, and seeding it leaves AXIOM's leads unchanged.

Second, four further runs retrained every policy on fresh seeds, 10 to 40 per setting, and 81 of their 90 pre-stated checks held. The nine checks that failed concern secondary margins rather than any headline result. On fresh seeds the DQN baselines reached up to 83.3% compliance at seven gNBs, above the 80% bound we had set, and AXIOM's small lead over DQN-SA in Env A at three gNBs did not recur. A fifth fresh-seed run held all three of its checks at thirty-seven gNBs on 20 seeds. A sixth held all four in Env B at nineteen gNBs on 15 seeds, as Section S7 of Online Resource 1 reports. A final pre-registered run held eight of its nine checks. It replicated the transfer and load-shift results on fresh seeds, and the one check that failed, at three gNBs past the tuning load, is reported in Section 6.4. Section S2 of Online Resource 1 compares each reported difference from a learned baseline with its fresh-seed counterpart, and 35 of the 40 whose CI excludes zero hold. The pre-registered extension that doubled the main samples also served as a replication. On its new seeds alone every headline lead recurred except two secondary ones, AXIOM's lead over DQN-SA at seven gNBs on the ring layout and its compliance lead over the tuned neighbour-aware rule in Env B at three gNBs, and both still hold in the pooled samples.

Third, a script written independently of the analysis pipeline, with its own log reader and Holm correction, recomputes the numbers in every results table and the key results sentences from the raw logs. In the current version, all 663 of its checks pass. Section S2 of Online Resource 1 and Online Resource 2 add interquartile means and probabilities of improvement [2], and all 51 orderings that this paper draws from means hold under them.

Table 6 Design and model complexity at 3, 7, 19, and 37 gNBs. Parameters used at execution, greedy inference time per control step on one central processing unit (CPU) thread, and median training time per seed at 3, 19, and 37 gNBs, on the ring layout for AXIOM

Policy	Design	Parameters	ms per step	Training s
AXIOM	Shared encoder, per-slice heads	7,782 at every size	1.04 / 1.98 / 4.79 / 9.89	624 / 892 / 1182
DQN-SA	One flattened controller	21,250 / 26,370 / 41,730 / 64,770	0.61 / 1.42 / 3.95 / 8.02	85 / 202 / 358
DQN-I	One learner per gNB	55,302 / 129,038 / 350,246 / 682,058	0.59 / 1.38 / 3.73 / 8.08	181 / 1054 / 2088
MAPPO	Shared actor, central critic	5,122 at every size	0.19 / 0.20 / 0.22 / 0.27	73 / 256 / 495

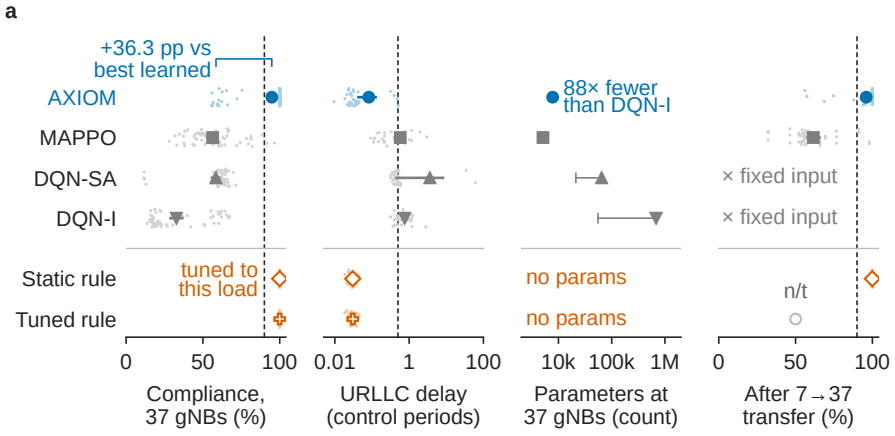
6.7 Federation, Privacy, Disruption, and Live Deployment

Federated training costs AXIOM little reward per step on average. Pooled over all 30 federated seeds, ten of them fresh, the cost is +0.491 per step with 95% CI [−0.08, 1.09] and $p = 0.14$. Even so, a cost of up to about one reward per step cannot be excluded. Per-step DP-SGD noise lowers reward by 0.77 to 1.25 per step without a significant drop, but its formal guarantee is weak, with ε of about 2,600 or more. Under gNB dropout and agent churn every policy loses reward significantly, and no gap between AXIOM and any baseline survives Holm correction at ten seeds. Demand spikes leave AXIOM's decisions largely intact, with block precision at 0.98, and Section S5 of Online Resource 1 gives the protocol. A live single-gNB OAI testbed built on ORANSlice [11] confirmed the control path that AXIOM would use, with 0.000% loss at one and at two UEs per slice. It never ran short of capacity, however, as Section S3 of Online Resource 1 reports.

7 Discussion and Limitations

No learned baseline earns significantly more reward than AXIOM in any setting we tested, and its margin over the cheaper MAPPO is largest under time-varying load, coupling, and uneven per-gNB load. A rule known in advance, however, matches or beats AXIOM in steady conditions at every size, and AXIOM beats a tuned rule only when the load moves beyond the rule's tuning conditions. Its practical case is for operators who cannot hand-tune and re-tune rules as conditions change, rather than for a fixed, well-understood cluster. Fig. 9 sets out what each controller offers. Among the learned controllers, only AXIOM meets both service thresholds at 37 gNBs and keeps above 90% compliance when moved there from seven gNBs without retraining. AXIOM learns its own shedding thresholds from the reward, close to those an engineer would tune by hand, and one 7,782-parameter network trained at seven gNBs serves clusters five times larger. The static rule must know in advance which slice to shed, and the tuned rule needs a fresh threshold search whenever conditions change. AXIOM is not retrained after deployment either, so its edge under shifted load comes from what it learned rather than from online adaptation. Fig. 10 maps this boundary over every cluster size and load we tested. In Env B AXIOM pulls ahead of the tuned rule at every size from 110% of the rule's tuning load, and from 105% at nineteen gNBs. In Env A it never significantly beats the static rule at any size or load. In Env B the static rule also pulls ahead of the tuned rule past its tuning load. At nineteen gNBs it leads the tuned rule by 8.05 to 10.63 reward per step, in all 15 seeds at Holm-adjusted $p < 0.001$, and its lead of 1.8 to 2.0 over AXIOM stays within chance, at Holm-adjusted $p \geq 0.08$.

Several limitations bound the findings of this paper. First, both of our simulators are synthetic rather than measured. They use a first-order coupling model with no radio channel, user mobility, or measured traffic traces, and the live testbed never reached contention. External validity on hardware under load therefore remains the main open question. Future work could drive demand from public cellular activity records, such as the Milan dataset [7], to test AXIOM under measured load. Second, we did not reimplement a published slice-admission algorithm, so our learned baselines are



b

	AXIOM	MAPPO	DQN-SA	DQN-I	Static rule	Tuned rule
Transfers without retraining	✓ 95–96%	◐ 61.5%	× fixed input	× fixed input	✓ size-free	✓ 7→19, Env B
Beats tuned rule past its load	✓ +7.5/step	× 0 of 9 wins	◐ 4 of 9 wins	◐ 3 of 9 wins	✓ +9.4/step	reference
Best under steady load	× −3.8/step	× −25.0/step	× −9.4/step	× −12.9/step	✓ best in A	✓ best in B
Private training	✓ FedAvg	× pooled	× pooled	✓ local	✓ no model	✓ no model

Fig. 9 What each controller offers. a, Env A at 37 gNBs, with means, 95% bootstrap CIs, and single seeds as dots. Dashed lines mark 90% compliance and half a control period of delay, and n/t marks a value not tested. b, Four capabilities, met, partly met, or not met. Reward gaps are per step against the tuned rule in Env B at 19 gNBs. Private training is measured for AXIOM and follows from the training design for the others

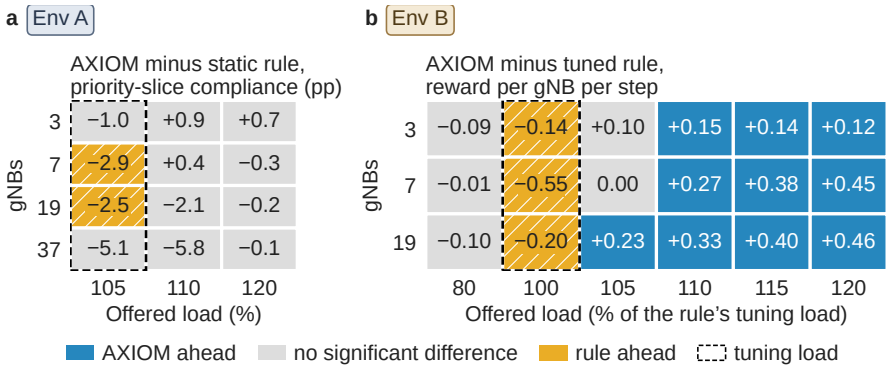


Fig. 10 Where AXIOM beats or trails a rule tuned once at nominal load, paired per seed. a, Env A priority-slice compliance against the static rule. b, Env B reward per gNB against the tuned rule. Filled cells differ at Holm-adjusted $p < 0.05$, hatched where the rule leads. The dashed box marks the rule's tuning load

standard multi-agent methods and ablations of AXIOM rather than prior admission controllers. Third, attention added little over a per-node encoder, at most 0.068 reward per step, and a value-decomposition baseline matched AXIOM at three gNBs, so AXIOM's contribution is a design that scales rather than a new learning principle. That per-node variant shares its parameters across gNBs but passes no messages between them, so the gain at scale comes from parameter sharing and per-slice heads. Fourth, the reward charges each slice's SLA penalty once for the cluster, and AXIOM's compliance at seven gNBs depends on that choice. Charging the penalty per violating gNB raised its priority-slice compliance by 10.7 points in Env B at seven gNBs on the ring layout, at Holm-adjusted $p = 0.007$. At nineteen gNBs on the hex layout it raised AXIOM's per-gNB compliance by only 1.9 points, at $p = 0.65$. Fifth, the load-shift comparison re-tunes rules only offline on shifted training streams, and a rule that re-tunes itself online was not tested. Finally, the pre-registrations were committed to the authors' own version-control history rather than an external registry, and development work that changed no reported result is disclosed in Section S4 of Online Resource 1.

8 Conclusion

This paper presented AXIOM, a multi-agent slice admission controller with a shared encoder and per-slice heads, evaluated under a protocol designed to make its results trustworthy. AXIOM's advantage over learned controllers grows with the cluster. Under static load it reaches 99.2% of the static shed-mMTC rule's reward at three gNBs and keeps priority-slice compliance between 94.9% and 97.4% from seven to thirty-seven gNBs, while every learned baseline stays below 84%. At thirty-seven gNBs its lead over the best learned baseline reaches 36.3 points, and 36.5 points on 20 fresh seeds, with 7,782 parameters against up to 682,058.

Under time-varying load and coupling AXIOM earns 26% to 36% more reward than MAPPO at three and nineteen gNBs, so its extra cost pays most where the environment is dynamic. When load rises 10% to 20% above the conditions a tuned rule was set for, AXIOM overtakes that rule at nineteen gNBs by 6.2 to 8.7 reward per step without retraining. Simple rules still set the bar in other settings. A static rule known in advance does better under steady load and at nineteen gNBs under coupling, and a tuned rule beats every learned policy at its tuning conditions, as Section 7 discusses. The main findings held on fresh seeds never used before.

More broadly, the evaluation safeguards show that learned admission control should be judged against trivial and tuned rules. Its metrics should be tied to the objective, and its results should hold on independent seeds under a pre-registered analysis.

Three directions for future work follow from these limits. First, extending our live single-gNB deployment [25] to multiple gNBs would test AXIOM's gains under real contention. Second, giving the learner the queue-threshold structure that the tuned rules exploit may close the remaining gap to them. Finally, adding noise to the per-round model updates rather than to every step would make the formal privacy guarantee meaningful.

Supplementary information. Online Resource 1 is a PDF that gives the full verification results, the live testbed check, additional setup details, and additional

results. Online Resource 2 is a CSV file that lists, for every setting and policy, the interquartile means of reward and of priority-slice compliance and the probability that AXIOM improves on each baseline, each with its 95% CI.

Acknowledgements. This work was supported by the Fundamental Research Grant Scheme (FRGS) of the Ministry of Higher Education (MOHE), Malaysia, under grant FRGS/1/2024/ICT06/TAYLOR/02/1.

Statements and Declarations

The following statements should be understood to apply to this manuscript unless otherwise indicated.

- Funding: This work was supported by the Fundamental Research Grant Scheme (FRGS), Ministry of Higher Education (MOHE), Malaysia, under grant number FRGS/1/2024/ICT06/TAYLOR/02/1.
- Competing interests: The authors declare no competing interests.
- Ethics approval and consent to participate: Not applicable, since this study involves no human participants, animal subjects, or identifiable patient data. All reported results are either offline simulation campaigns or measurements from a laboratory radio testbed.
- Consent for publication: Not applicable, since this manuscript contains no personal data or identifying information about any individual.
- Data availability: The experiment logs, checkpoints, and campaign result files underlying this paper's findings are available from the corresponding author on reasonable request. They are not yet publicly archived because the research is ongoing and the dataset is not yet complete.
- Materials availability: Not applicable, since this study involves no physical or biological materials.
- Code availability: The framework and experiment scripts used to produce this paper's results, including the reproducibility manifest and analysis scripts used to independently re-derive every table and figure in this paper, are available from the corresponding author on reasonable request.
- Use of artificial intelligence tools: Anthropic's Opus agent helped write the scripts and edit the text, and the authors checked its outputs and take full responsibility.
- Author contributions: M. P. Kunasegran: conceptualisation, methodology, software, investigation, writing (original draft). W. L. Pang: supervision, writing (review and editing). S. K. Phang: supervision, writing (review and editing).

References

[1] Abadi M, Chu A, Goodfellow I, McMahan HB, Mironov I, Talwar K, Zhang L (2016) Deep learning with differential privacy. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp 308–318, https://doi.org/10.1145/2976749.2978318

[2] Agarwal R, Schwarzer M, Castro PS, Courville A, Bellemare M (2021) Deep reinforcement learning at the edge of the statistical precipice. In: *Advances in Neural Information Processing Systems (NeurIPS)*, pp 29304–29320

[3] Ahmadi M, Moayyedi A, Sulaiman M, Salahuddin MA, Boutaba R, Saleh A (2024) Generalizable 5G RAN/MEC slicing and admission control for reliable network operation. *IEEE Transactions on Network and Service Management* 21(5):5384–5399. https://doi.org/10.1109/TNSM.2024.3437217

[4] AlQahtani SA (2025) Dynamic slice allocation in 5G cloud RAN. *Wireless Personal Communications* 143(3-4):427–467. https://doi.org/10.1007/s11277-025-11835-y

[5] Amiri E, Wang N, Shojafar M, Tafazolli R (2024) Edge-AI empowered dynamic VNF splitting in O-RAN slicing: A federated DRL approach. *IEEE Communications Letters* 28(2):318–322. https://doi.org/10.1109/LCOMM.2023.3341302

[6] Asdikian JP, Amro A, Mehyeddine L, Natalino C, Sbeity I, Maier G, Monti P, Troia S, Ayoub O (2025) Verifying behavior of reinforcement learning agents for network slice admission control. In: *Proceedings of the 21st International Conference on Network and Service Management (CNSM)*, pp 1–6, https://doi.org/10.23919/CNSM67658.2025.11297450

[7] Barlacchi G, De Nadai M, Larcher R, Casella A, Chitic C, Torrisi G, Antonelli F, Vespignani A, Pentland A, Lepri B (2015) A multi-source dataset of urban life in the city of Milan and the province of Trentino. *Scientific Data* 2:150055. https://doi.org/10.1038/sdata.2015.55

[8] Bega D, Gramaglia M, Banchs A, Sciancalepore V, Costa-Pérez X (2020) A machine learning approach to 5G infrastructure market optimization. *IEEE Transactions on Mobile Computing* 19(3):498–512. https://doi.org/10.1109/TMC.2019.2896950

[9] Bouroudi A, Outtagarts A, Hadjadj-Aoul Y (2025) GNN-based multi-agent DRL for energy-efficient multi-domain 6G resource orchestration. In: *Proceedings of the IEEE Global Communications Conference (GLOBECOM)*, pp 4457–4462, https://doi.org/10.1109/GLOBECOM59602.2025.11432814

[10] Bun M, Steinke T (2016) Concentrated differential privacy: Simplifications, extensions, and lower bounds. In: *Theory of Cryptography Conference (TCC)*, pp 635–658, https://doi.org/10.1007/978-3-662-53641-4_24

[11] Cheng H, D'Oro S, Gangula R, Velumani S, Villa D, Bonati L, Polese M, Melodia T (2024) ORANSlice: An open source 5G network slicing platform for O-RAN. In: *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (ACM MobiCom)*, pp 2297–2302, https://doi.org/10.1145/3636534.3701544

[12] Doanis P, Spyropoulos T (2024) Sample-efficient multi-agent DQNs for scalable multi-domain 5G+ inter-slice orchestration. *IEEE Transactions on Machine Learning in Communications and Networking* 2:956–977. https://doi.org/10.1109/TMLCN.2024.3420268

[13] Doherty M, Matzner R, Sadeghi R, Bayvel P, Beghelli A (2025) Reinforcement learning for dynamic resource allocation in optical networks: Hype or hope? *Journal of Optical Communications and Networking* 17(9):D1. https://doi.org/10.1364/JOCN.559990

[14] Fatehi K, Ghourtani MR, Sonee A, Yadav P, Russo AM, Ahmadi H, Calinescu R (2026) Interpretable attention-based multi-agent PPO for latency spike resolution in 6G RAN slicing. In: *Proceedings of the IEEE International Conference on Communications (ICC)*, pp 1–6, https://doi.org/10.1109/ICC59461.2026.11586913

[15] Gong Y, Gong P, Jiang H, Xie W, Wang C, Xu P (2026) Intelligent resource allocation for multiaccess edge computing in 5G ultra-dense slicing network using federated multiagent DDPG algorithm. *China Communications* 23(1):273–289. https://doi.org/10.23919/JCC.fa.2022-0187.202601

[16] Hao X, She C, Yeoh PL, Liu Y, Vucetic B, Li Y (2024) Hybrid-task meta-learning: A GNN approach for scalable and transferable bandwidth allocation. *IEEE Transactions on Wireless Communications* 23(12):19820–19835. https://doi.org/10.1109/TWC.2024.3487533

[17] Henderson P, Islam R, Bachman P, Pineau J, Precup D, Meger D (2018) Deep reinforcement learning that matters. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, https://doi.org/10.1609/aaai.v32i1.11694

[18] Kamal YB, Abdulkafi AA (2026) Reinforcement learning double deep Q-network techniques for NextG network. *Wireless Personal Communications* 146(6):3175–3197. https://doi.org/10.1007/s11277-026-12048-7

[19] Khani M, Jamali S, Sohrabi MK, Sadr MM, Ghaffari A (2024) Slice admission control in 5G cloud radio access network using deep reinforcement learning: A survey. *International Journal of Communication Systems* 37(13):e5857. https://doi.org/10.1002/dac.5857

[20] Krishnamoorthy R, Gireesh Babu CN, Gite P, Khadidos AO, Khadidos AO, Mirza OM, Selvarajan S (2026) Federated learning for dynamic resource allocation in 6G network slicing. *Wireless Personal Communications* https://doi.org/10.1007/s11277-026-11943-3

[21] Kulkarni D, Venkatesan M, Kulkarni AV (2024) Actor critic based reinforcement learning for joint resource allocation and throughput maximization in 5G RAN slicing. *Wireless Personal Communications* 138(1):623–640. https://doi.org/10.

1007/s11277-024-11526-0

[22] Kunasegran MP, Pang WL, Phang SK (2025) Optimising resource allocation in 5G networks: Balancing URLLC and eMBB traffic under gNB congestion. In: *2025 Multimedia University Engineering Conference (MECON)*, pp 1–6, https://doi.org/10.1109/MECON67253.2025.11277096

[23] Kunasegran MP, Pang WL, Phang SK (2025) SLA-aware DRL for joint slice admission control and load balancing in multi-service 5G RAN. In: *2025 9th International Conference on Recent Advances and Innovations in Engineering (ICRAIE)*, pp 207–212, https://doi.org/10.1109/ICRAIE65839.2025.11239148

[24] Kunasegran MP, Pang WL, Phang SK (2026) Comprehensive review of optimization techniques for user-centric distributed network slicing in 5G networks. *IEEE Access* 14:114630–114648. https://doi.org/10.1109/ACCESS.2026.3716911

[25] Kunasegran MP, Pang WL, Phang SK (2026) DQN-QoE slice admission control for live OAI Open RAN. In: *2026 International Automatic Control Conference (CACS 2026)*, Taichung, Taiwan, to appear in IEEE Xplore

[26] Li F, Hao Y, Yang S, Zhao P (2025) OACR^2: Online admission control and resource reservation for 5G slice networks with deep reinforcement learning. *IEEE Transactions on Mobile Computing* 24(8):7360–7376. https://doi.org/10.1109/TMC.2025.3548767

[27] Li Y, Wang H, Min G, Wang X, Wang C (2026) A flexible and scalable multi-agent learning framework for dynamic RAN slicing in 6G native-AI networks. *IEEE Transactions on Mobile Computing* 25(5):7258–7273. https://doi.org/10.1109/TMC.2025.3642213

[28] Lim WYB, Luong NC, Hoang DT, Jiao Y, Liang YC, Yang Q, Niyato D, Miao C (2020) Federated learning in mobile edge networks: A comprehensive survey. *IEEE Communications Surveys & Tutorials* 22(3):2031–2063. https://doi.org/10.1109/COMST.2020.2986024

[29] Ming Z, Yu H, Taleb T (2024) Federated deep reinforcement learning for prediction-based network slice mobility in 6G mobile networks. *IEEE Transactions on Mobile Computing* 23(12):11937–11953. https://doi.org/10.1109/TMC.2024.3404125

[30] Mohajer A, Mirzaei A, Darabi M, Fernando X (2026) Joint SLA-aware task offloading and adaptive service orchestration with graph-attentive multi-agent reinforcement learning. *IEEE Transactions on Network and Service Management* 23:3737–3754. https://doi.org/10.1109/TNSM.2026.3673188

[31] Ojijo MO, Ramotsoela D, Oginga RA (2024) Slice admission control in 5G wireless communication with multi-dimensional state space and distributed action space:

A sequential twin actor-critic approach. *Computer Networks* 255:110878. https://doi.org/10.1016/j.comnet.2024.110878

[32] Phyu HP, Naboulsi D, Stanica R (2025) ICE-CREAM: Multi-agent fully cooperative decentralized framework for energy efficiency in RAN slicing. *IEEE Transactions on Network and Service Management* 22(2):1859–1873. https://doi.org/10.1109/TNSM.2024.3524503

[33] Polese M, Bonati L, D'Oro S, Basagni S, Melodia T (2023) ColO-RAN: Developing machine learning-based xApps for Open RAN closed-loop control on programmable experimental platforms. *IEEE Transactions on Mobile Computing* 22(10):5787–5800. https://doi.org/10.1109/TMC.2022.3188013

[34] Polese M, Bonati L, D'Oro S, Basagni S, Melodia T (2023) Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges. *IEEE Communications Surveys & Tutorials* 25(2):1376–1411. https://doi.org/10.1109/COMST.2023.3239220

[35] Rashid T, Samvelyan M, Schroeder de Witt C, Farquhar G, Foerster J, Whiteson S (2020) Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research* 21(178):1–51

[36] Raza MR, Natalino C, Öhlen P, Wosinska L, Monti P (2019) Reinforcement learning for slicing in a 5G flexible RAN. *Journal of Lightwave Technology* 37(20):5161–5169. https://doi.org/10.1109/JLT.2019.2924345

[37] Shao Y, Li R, Hu B, Wu Y, Zhao Z, Zhang H (2021) Graph attention network-based multi-agent reinforcement learning for slicing resource management in dense cellular network. *IEEE Transactions on Vehicular Technology* 70(10):10792–10803. https://doi.org/10.1109/TVT.2021.3103416

[38] Sirma M, Kavak A, Inner AB (2025) Optimizing relay selection in D2D communication for next-generation wireless networks using multi-agent reinforcement learning: A novel approach. *Wireless Personal Communications* 140(3-4):945–969. https://doi.org/10.1007/s11277-025-11753-z

[39] Sulaiman M, Moayyedi A, Ahmadi M, Salahuddin MA, Boutaba R, Saleh A (2023) Coordinated slicing and admission control using multi-agent deep reinforcement learning. *IEEE Transactions on Network and Service Management* 20(2):1110–1124. https://doi.org/10.1109/TNSM.2022.3222589

[40] Sunehag P, Lever G, Gruslys A, Czarnecki WM, Zambaldi V, Jaderberg M, Lanctot M, Sonnerat N, Leibo JZ, Tuyls K, Graepel T (2018) Value-decomposition networks for cooperative multi-agent learning based on team reward. In: *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, pp 2085–2087, https://doi.org/10.65109/jsrc7365

[41] Tao Z, Xu W, You X (2025) Digital twin-accelerated online deep reinforcement learning for admission control in sliced communication networks. *IEEE Transactions on Communications* 73(4):2490–2504. https://doi.org/10.1109/TCOMM.2024.3476430

[42] Tashman DH, Cherkaoui S (2026) Adversarial attacks in AI-driven RAN slicing: SLA violations and recovery. In: *Proceedings of the International Wireless Communications and Mobile Computing Conference (IWCMC)*, pp 1391–1396, https://doi.org/10.1109/IWCMC69287.2026.11580033

[43] Tsampazi M, D'Oro S, Polese M, Bonati L, Poitau G, Healy M, Melodia T (2023) A comparative analysis of deep reinforcement learning-based xApps in O-RAN. In: *Proceedings of the IEEE Global Communications Conference (GLOBECOM)*, pp 1638–1643, https://doi.org/10.1109/GLOBECOM54140.2023.10437367

[44] Tsampazi M, D'Oro S, Polese M, Bonati L, Poitau G, Healy M, Alavirad M, Melodia T (2025) PandORA: Automated design and comprehensive evaluation of deep reinforcement learning agents for Open RAN. *IEEE Transactions on Mobile Computing* 24(4):3223–3240. https://doi.org/10.1109/TMC.2024.3505781

[45] Wu Z, Pan S, Chen F, Long G, Zhang C, Yu PS (2021) A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems* 32(1):4–24. https://doi.org/10.1109/TNNLS.2020.2978386

[46] Yasin TV, Yu CM, Wang LC (2025) Differential privacy federated edge learning-assisted for securing RAN intelligent controller in O-RAN 6G communications. In: *Proceedings of the IEEE VTS Asia Pacific Wireless Communications Symposium (APWCS)*, pp 1–5, https://doi.org/10.1109/APWCS67981.2025.11151868

[47] Yu C, Velu A, Vinitsky E, Gao J, Wang Y, Bayen A, Wu Y (2022) The surprising effectiveness of PPO in cooperative multi-agent games. In: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, pp 24611–24624, https://doi.org/10.52202/068431-1787

[48] Zangooei M, Golkarifard M, Rouili M, Saha N, Boutaba R (2024) Flexible RAN slicing in Open RAN with constrained multi-agent reinforcement learning. *IEEE Journal on Selected Areas in Communications* 42(2):280–294. https://doi.org/10.1109/JSAC.2023.3336156

[49] Zhang H, Zhou H, Erol-Kantarci M (2022) Federated deep reinforcement learning for resource allocation in O-RAN slicing. In: *Proceedings of the IEEE Global Communications Conference (GLOBECOM)*, pp 958–963, https://doi.org/10.1109/GLOBECOM48099.2022.10001658